Source: docs\CUSTOMER_AUTH.md | Export: 11 November 2026

Customer authentication

This milestone implements real Firebase customer authentication using the existing `parinaya-silks` app and `(default)` Firestore database. Local tests use loopback emulators. No production customers, permissions, rules, configuration or deployments were changed.

Current update: admin-managed login portraits and required registration mobile contact. New profile creation requires the reviewed mobile-number rules; previous email/password-only rule-publication notes below describe that earlier change.

The subsequent account-linked Bag and Wishlist milestone reuses this authentication and safe-return flow. Its shopping permissions remain separate from administrator permissions.

Routes and session behavior

Route	Behavior
`/login`	Email/password sign-in, password visibility, Remember me, reset and registration links
`/register`	Name, email, mobile number, password/confirmation; Firebase password-policy validation; registration, profile creation and immediate sign-in
`/forgot-password`	Neutral password-reset confirmation for existing and unknown email addresses
`/verify-email`	Legacy account alias only; no verification flow
`/account`	Real name/email/mobile contact, profile recovery, browse and sign-out
`/account/orders`	Same minimal account view; order history is not implemented

The Account navigation entry preserves the current implemented browsing URL for signed-out visitors. Explicit `?returnTo=` values are allowlisted internal routes; external URLs, admin/API paths, recursive auth links, traversal and unsafe encodings are rejected. The default is `/account`. Completed registration opens that account or the validated internal destination. A failed name/profile save opens `/account` for recovery instead. Signed-in visitors to login/registration are redirected without replacing their session.

Customer and administrator routes share **one Firebase Auth instance**. Initialization restores existing local/session persistence without calling `setPersistence` or signing anyone out. Explicit customer login selects local persistence when Remember me is checked, and session persistence otherwise, before authenticating. Registration selects session persistence and validates mobile contact before Auth creation. Firebase owns its authentication storage; the application never manually stores passwords or tokens. Sign-out affects the shared session, including administrator access in the same browser.

Initial restoration has a neutral loading state. Token updates do not trigger a full-page loading overlay. Email verification is not required or sent. Registration and account loading do not request verification, refresh verification status or change Firebase verification flags. Existing unverified customers can sign in and use implemented customer features. Bag, favourites, orders, checkout and payments remain outside this milestone.

Private profile schema and recovery

`customerProfiles/{Firebase Auth UID}` uses the following fields:

```
uid: string matching the document ID and authenticated UID
displayName: trimmed, nonempty string, maximum 100 characters
mobileNumber: normalized E.164 string required for new profiles; optional on existing profiles
createdAt: Firestore server timestamp
updatedAt: Firestore server timestamp
```

The older business `customers` collection has no connected customer identity repository. The new collection keeps private authentication profiles independent of that future business module. No passwords, tokens, role, admin or verification flags are written.

A transaction creates a missing profile or returns the existing profile without overwriting it. Repeated login does not update timestamps or replace an existing name. Profile editing/deletion is not part of this release. If Auth registration succeeds but saving the name/profile fails, the account remains valid and opens the account screen. Loading retries the idempotent profile operation; persistent failures offer explicit profile recovery. Customers must not register again. Recovery also works after a later sign-in using the saved Auth display name, or a supplied name if that save failed.

The new Firestore match permits only owner `get` and validated owner `create`, requires both timestamps to equal `request.time`, and denies list/update/delete. Other users, including administrators, cannot read someone else's private profile through the browser SDK. Privileged fields, foreign UIDs, client timestamps and customer writes to admin/business repositories are rejected. The existing administrator checks still require **both** `admin: true` and enabled `adminAccess/{uid}`; customer registration creates neither.

Owner setup before production acceptance

1. Confirm Email/Password is enabled in Firebase Console → Authentication → Sign-in method for **`parinaya-silks`**. Review the configured password policy. Registration calls the SDK's `validatePassword` against that policy and fails safely if the policy cannot be retrieved; no hardcoded production policy or fallback is used.
2. In Authentication → Settings → Authorized domains, ensure **`parinayasilks.vercel.app`** is present. Add only preview/custom domains you intend to use for real email flows. Add `localhost` only if you intentionally test the real backend locally; new Firebase projects may not include it automatically. Emulator-only tests need no console changes. See Firebase password authentication and authorized-domain guidance.
3. Review the password-reset email template, sender and delivery. Keep Firebase's default hosted email action handler, normally `https://parinaya-silks.firebaseapp.com/__/auth/action`. Password reset supplies the same-origin continue URL `/login` with `handleCodeInApp: false`; the hosted handler consumes the reset code first. No verification email template or action is used by the customer flow. `/verify-email` is only a legacy account alias. See Firebase email action state.
4. **Removing customer verification requires no Firestore rule change or new rule publication.** Existing rules already authorize profiles by UID without requiring verified email. If the original restrictive profile rules have never been published, that earlier setup requirement remains: compare deployed rules with `firestore.rules`, preserve legitimate intervening changes and publish the reviewed rules. A broad permissive starter rule would override profile isolation because allow rules are additive. The original owner publication command is:

   ```
   npx.cmd firebase deploy --only firestore:rules --project parinaya-silks
   ```

   This command has **not** been run. No new index is needed for a UID document transaction. Existing catalog indexes are unchanged.
5. Commit/push the reviewed application and tests through the existing Vercel workflow. No new environment variable, credential or dependency is required. Keep Admin SDK/Cloudinary secrets server-only; never add them to `VITE_*` variables.

Deployed rules, authorized domains, actual password policy, sender delivery and hosted action handling must be verified by the owner. Local emulator success does not establish production readiness.

Local validation and acceptance

```
npm.cmd run lint
npm.cmd test
npm.cmd run build
npm.cmd run test:catalog-runtime
npm.cmd run test:admin
```

`test:admin` starts isolated Auth/Firestore emulators at `127.0.0.1:9099` and `127.0.0.1:8080`, the test Vite app at `127.0.0.1:3100`, test API at `3200`, missing-configuration app at `3101`, and local production preview at `4200`. It creates synthetic emulator users only and runs profile rules, real SDK customer browser flows and existing admin/category/product suites. Java and host Python Playwright/Chromium are required. Synthetic email action codes remain inside the test process; CLI email links are redacted before logging.

The optional packaged API check requires the pinned builder in ignored storage. If absent, install it with `npm.cmd install --prefix .cache/vercel-runtime --no-save --ignore-scripts --no-audit --no-fund @vercel/node@24.0.3 @vercel/build-utils@14.21.3`. The runtime check invokes the packaged function without a TypeScript loader; it does not call a deployed endpoint or need credentials.

Password-policy test limitation: Firebase CLI 15.34.0 does not implement `getPasswordPolicy`. The browser fixture intercepts only the exact loopback policy endpoint, returning an explicit policy (12 characters, uppercase/lowercase/number/symbol). The real SDK validator is exercised; account creation, password authentication, persistence, email actions and Firestore transactions use real emulators. Production policy retrieval is not stubbed and remains a production acceptance check.

For ordinary local UI acceptance, run `npm.cmd run dev` on `http://localhost:3000`, then `npm.cmd run preview -- --host 127.0.0.1 --port 4173` after building. These ordinary modes use configured Firebase, so do not create synthetic accounts there. With both servers running, `python scripts/storefront-browser.test.py` verifies public empty states, blocked business actions, invalid auth forms and routing with backend calls intercepted. Use the emulator suite for account creation and email action testing.

Browser coverage includes 320/375/390/768/1024/1440px, one form/unique IDs, portrait screenshots, no overflow, visible active Account tab, touch/input sizing, resize preservation, short-screen reflow/scrolling, keyboard submit, password visibility, menu/search/legal links, invalid inputs/policy, duplicate submit, incorrect credentials, network error, profile failure/recovery, real password-reset email actions, absence of verification requests/prompts, unchanged false verification flags, immediate registration destinations, clean account spacing, session/local persistence and new tabs, safe returns, sign-

out and customer admin denial. Rules cover profile ownership and privilege escalation. Existing admin coverage verifies the same administrator session survives a customer login-page visit without changing its persistence. Physical phone keyboard/safe-area behavior and native browser zoom still require device acceptance; short-viewport/reflow checks are their local approximation.

Email/password-only update validation: TypeScript check, 52 unit tests, production build, all four emulator rule/service suites and customer/admin/category/product browser suites passed. Customer checks confirmed no verification requests or emulator verification email actions, unchanged unverified status, real password-reset email actions, profile failure/recovery, registration destinations, persistence and clean account layouts at all six widths. No hosted authentication or deployed-rule verification was performed. The production build retains its large-chunk advisory.

Vercel acceptance after owner release

Use an owner-approved real test account; automated tests do not create production users.

1. Open `/login` at the production domain; compare portrait/form at desktop and mobile widths. Resize with filled fields, use Tab/Enter, toggle password visibility and verify menu/search/legal/Continue browsing and bottom navigation. Test native zoom and the on-screen keyboard on an actual phone; all controls must remain reachable by scrolling.
2. Register once and confirm immediate sign-in and account/validated return navigation. Check the name and UID-keyed private profile/timestamps. No verification email, reminder, field or action should appear. Sign in with an existing unverified account; its Firebase verification flag must remain unchanged. If profile saving fails, use account recovery instead of registering again.
3. Sign out, enter incorrect credentials, then sign in successfully with Remember me unchecked. Reload and open a new tab to verify session scope. Repeat checked and verify remembered restoration. Opening `/login` while signed in should return to the account or a validated destination without a sign-in flash.
4. Open Account from Shop, sign in and confirm the original Shop URL is restored. Try an external `returnTo` and confirm `/account`. Sign out and confirm private details disappear immediately.
5. Request a reset for the approved address and an unknown address; both get neutral confirmation. Complete the real email reset action and sign in with the changed password. Never share that password or action URL in logs/chat.
6. Confirm an ordinary customer cannot open `/admin/products`, read another private profile, list profiles or write privileges. Confirm the existing authorized administrator still reaches admin/category/product screens and public catalog browsing still works. Never create products merely to test authentication.

Design assets and implementation map

The reference source is the owner-supplied **Parinaya Silks Desktop Login Page.png** (1586×992). `public/assets/customer-auth-portrait.jpg` is only its portrait rectangle `(x=0, y=77, width=818, height=915)`, JPEG quality 94 (~252 KB). No screenshot form, text or header is included. Desktop uses intentional centered cover; mobile uses top-aligned cover to preserve the face. The 1024px CSS breakpoint keeps a readable single column on tablets before the desktop split. Mobile content scrolls naturally rather than shrinking to one screen.

The subtle botanical SVG is decorative code artwork, not a pixel-exact tracing. The PS mark reuses the site's serif treatment; headings use the existing Bodoni Moda family and form controls use system sans-serif. No local font files were available. The existing shared header/menu/search and five-tab navigation are reused. Typography, exact botanical strokes and scalable spacing therefore differ slightly from the composite references.

Files: `src/customer/` owns shared screens, styles, policy, session, service and transaction logic; `src/services/firebase/auth.ts` shares Auth initialization with `adminAuth.ts`; five existing account/auth views delegate to these screens. `App.tsx`, `Header.tsx` and `MobileBottomNav.tsx` connect routing and responsive shell behavior. `firestore.rules` adds private profile access. `scripts/customer-*`, existing browser suites, emulator suite and `package.json` provide regressions; `firebase-test-cli.cjs` redacts synthetic email action links. Portrait and botanical assets live under `public/assets/`.

Email/password-only update files

`src/customer/service.ts` removes verification sends, resend cooldown and verification refresh. `CustomerAuth.tsx` removes verification-specific context state; `CustomerAccount.tsx` keeps name/email, profile recovery, Browse sarees and Sign out. `CustomerAuthScreen.tsx` navigates after registration and retains account recovery; `src/views/VerifyEmailView.tsx` preserves old bookmarks as an account alias. `scripts/customer-browser.test.py` checks no verification requests/prompts, unchanged unverified status, reset emails, registration destinations and account layouts. `customer-fixture-control.ts` reads only the emulator verification boolean in addition to its existing safe inspection fields. Administrator authorization, profile rules, existing layouts and backend catalog code are unchanged.